COMPSCI 701:
Creating Maintainable Software

Lecture 9.1: SOLID and Alterability

Ewan Tempero, Yu-Cheng Tu
School of Computer Science

Agenda

- Admin
  - ?
- Context reuse
- The SOLID Design Principles
- How SOLID affects Alterability

Previously in COMPSCI701

- There isn't a model for alterability so we will create our own — Change Ratio (CR= $\frac{C}{N}$)
- For a given change case with impact I ...
  - Determine the number of "classes" (files) of the existing design (N)
  - Determine the number of those classes (files) that need to be changed while implementing the change case (C)
  - Compare CR=$\frac{C}{N}$ to I
    - If $\frac{C}{N} \approx I$, then the "cost" of implementing this change case is about what is expected
    - If $\frac{C}{N} > I$, then the "cost" of implementing this change case is higher than what we expect (so lower alterability)
    - If $\frac{C}{N} < I$, then the "cost" of implementing this change case is lower than what we expect (so higher alterability)

Context Reuse

```
// Variation on Lecture 7.1
public void printFile(File start) {
    System.out.println(start.longDetails());
}
public void printFile() {
    System.out.println("Current:" + _current.shortDetails());
    printFile(_root);
}
```

- Objects from any implementation of `File` can be passed to `printFile()`.
- $\Rightarrow$ `printFile()` behaves differently depending on the implementation does despite this method not changing!
- We get to reuse the `printFile()` code

  $\Rightarrow$ "context reuse"

  (A different use of "context" to context schema!)

Context Reuse

```
public abstract class BulkItem {
  ...
  public String getDescription() {
    return getSpecifics() + " in bulk";
  }
  protected abstract String getSpecifics();
  ...
}
```

```
public class RainbowSweetFizzies extends BulkItem {
  protected String getSpecifics() {
    return "Rainbow Sweet Fizzies";
  }
}
```

- `getDescription()` is the context whose behaviour is changed despite it (or the class that contains it) not changing

SOLID Principles

- Set of “principles” for good object-oriented design
- Most have existed in some form for some time
- Assembled by Robert C. Martin in around 2000, coined by someone else (Possibly Michael Feathers)
- Seems to now be associated with Agile Development (Possibly due to “Agile Software Development: Principles, Patterns, and Practices” Robert C. Martin, Prentice Hall, 2003.)
- In fact, they are really just consequences of doing object-oriented design “properly” $\Rightarrow$ if know how to do good OOD then don’t need the principles (but they’re an ok place to start)

SOLID Principles

Single Responsibility Principle (SRP) A class should have one, and only one, reason to change.

Open Closed Principle (OCP) You should be able to extend a class' behaviour without modifying it.

Liskov Substitution Principle (LSP) Child classes must be substitutable for their Parent classes.

Interface Segregation Principle (ISP) Make fine grained interfaces that are client specific.

Dependency Inversion Principle (DIP) Depend on abstractions, not on concretions.

Single Responsibility Principle (SRP)

- A class should have only one responsibility (whatever that is. . .)
- A class should have one, and only one, *reason to change*
  - Update: “Gather together the things that change for the same reasons. Separate those things that change for different reasons.”

    https://8thlight.com/blog/uncle-bob/2014/05/08/SingleReponsibilityPrinciple.html
- “change” means altering the source code of the class
- The kind of change we usually are interested in is addition, that is, keeping the original behaviour and adding something more
- SRP and comprehensibility — See Lecture 6.1
  - Perhaps “Single Concept Principle” would be a more useful name!

Responsibility-Driven Design

- Rebecca Wirfs-Brock, Brian Wilkerson, and Lauren Wiener “Designing Object-Oriented Software”, Prentice Hall 1990
- responsibility = an obligation to perform a task or know information
- Responsibility-Driven Design
  1. what responsibilities does the *system* have?
  2. what *roles* should have those responsibilities?
  3. which *objects* should play a give role?
- $\Rightarrow$ Not single “responsibility” per class (at least by this definition of responsibility)

SRP and Alterability

- The fewer reasons to change, the lower C, or the higher N, so the CR is lower—all things being equal
- Example:
  - Design A has classes $A_1, A_2, A_3, \ldots, A_N$. Design B has classes $B_1, A_3, A_4, \ldots, A_N$ — that is, B has $N-2$ classes the same as in A, and one class (B_1) that is the combination of A_1 and A_2, a total of $N-1$ classes.
  - Consider change case c_1 that requires A_1 to change for Design A, and so B_1 will change for Design B.

    Design A: $C = 1$, $\text{CR} = \frac{1}{N}$

    Design B: $C = 1$, so $\text{CR}(B) = \frac{1}{N-1} > \frac{1}{N} = \text{CR}(A)$

    $\Rightarrow$ Design A is more alterable than Design B for c_1
  - By the same argument, for change case c_2 that requires A_2 to change, Design A is more alterable than Design B for c_2
  - But, consider change case c_3 that requires both A_1 and A_2 to change, and so only B_1 for design B. Then $\text{CR}(A) = \frac{2}{N} > \frac{1}{N-1} = \text{CR}(B)$ (when $N > 2$)
  - However most of the time $\text{CR}(A) \leq \text{CR}(B)$ — remember alterability is determined over the lifetime of the software system

SRP and Design Patterns

- Many of the elements of a design pattern are there to do just one thing
- With decorator, each decorator class should have different reasons to change
- With composite, each leaf and composite should have different reasons to change
- With command, each concrete command should have different reasons to change
- ...

SRP Conclusion

- The “one responsibility” definition is not actionable — does not provide objective actions to demonstrate how to follow the principle, or even identify when the principle is not being followed
- The “one reason to change” or “group things that change for the same reason together” definitions are perhaps easier to action, because the reasoning can be done in terms of change cases

Open Closed Principle

- *"software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification"* Bertrand Meyer (inventor of the Eiffel Programming language and author of "Object-oriented software construction" 1988)
- Meyer discussed it in terms of "abstract parent classes" (interfaces) and their children — you can "extend the behaviour" of the parent class by having children inherit from the parent without changing the parent
  - (but really this is not changing the behaviour of the parent)
- Robert C. Martin "The Open-Closed Principle", C++ Report, January 1996 (and again in his book) re-interpreted it.
- Martin's examples demonstrate that the real power comes from judicious use of polymorphism, that is identifying the method to execute via dynamic dispatch
- Typically leads to more abstract entities that are made concrete either through extension or through supplying the concrete implementations as parameters (e.g. with *dependency injection*)

Example

```
public class BulkItemDisplay {
  private List<BulkItem> _itemsOnDisplay = ...;
  ...
  public void listItems() {
    for (BulkItem item: _itemsOnDisplay) {
      System.out.println(item.getDescription());
    }
  }
}
```

```
public class RainbowSweetFizzies extends BulkItem {
  protected String getSpecifics() {
    return "Rainbow Sweet Fizzies";
  }
}
public class PineappleLumps extends BulkItem {
  protected String getSpecifics() {
    return "Pineapple Lumps";
  }
}
```

- Any client that uses `BulkItem` can be given a `RainbowSweetFizzies` or `PineappleLumps`, which behaves differently to `BulkItem`, so it looks like `BulkItem` has changed behaviour
- Note: The parent class doesn't have to be abstract

OCP and Alterability

- OCP is about avoiding the need to change existing classes $\Rightarrow$ reducing $\mathbb{C}$

OCP and Design Patterns

- Many design patterns define elements that are never meant to change, but provide the foundation on which to build changed behaviour
- Factory method — different concrete creator classes change the behaviour of the Creator component to create different products $\Rightarrow$ different behaviour (Open) without changing Creator (Closed)
- Template method — the child class changes the behaviour (Open) of the parent class, which does not change (Closed). See `BulkItem`

OCP Conclusion

- Reasonably actionable:
  - when we change something, anything that needs to changed is not closed to change
  - when designing something, think about what variations are needed for the context schema

Liskov Substitution Principle

- Child classes must be substitutable for their Parent classes
- In most object-oriented languages, most of the time, an object of any child class can be sensibly used where an object of a parent was required
  - . . . provided any overridden methods “make sense”
- “. . . the objects of the subtype ought to behave the same as those of the supertype as far as anyone or any program using supertype objects can tell.”

  Liskov and Wing. 1994. *A behavioral notion of subtyping*. TOPLAS 1994
- Originally “Data Abstraction and Hierarchy”, Keynote address by Barbara Liskov at OOPSLA 1987.
- Provides context reuse — the context can remain unchanged but program behaviour changes
- A semantic notion that is simulated by a type (or runtime) system

  $\Rightarrow$ necessarily provides only a partial check

Example (anti)

```
public class Utility {
  public boolean isValidFormat(String money) {
    // ... details here
  }
}

public class Money extends Utility {
  public Money(String value) {
    if (!isValidFormat(value)) { // self call
      // report error etc
    }
  }
}
```

- when would it make sense to substitute a `Money` object for a `Utility` object?

What “substitution” means

```
class NewZealandMoney implements Money {...}
class AussieMoney implements Money {...}
class Client {
  public static void main(String[] args) {
    NewZealandMoney nzm = new NewZealandMoney(100, 0);
    whatDoIHave(nzm);
  }
  private static void whatDoIHave(Money money) {
     // Money object expected, but NewZealandMoney and AussieMoney
     // can be substituted

     // Because Money understands formalString(), and NewZealandMoney
     // and AussieMoney also understand formalString(), the substitution
     //  will  work.
    System.out.println("I have " + money.formalString());
  }
}
```

- For the substitution to be useful, the substituted object must be used in the same way that the expected object is used

Where substitution is not useful

```
class Client {
  public static void main(String[] args) {
    // Substituting NewZealandMoney for Money! (via assignment)
    // Since Money != NewZealandMoney it is a "substitution", but only because Java is statically typed
    Money money = new NewZealandMoney(100, 0);
    whatDoIHave(money);

    // Also a substitution, but provides no value as nothing
    // can be done with moreMoney
    Object moreMoney = new NewZealandMoney(100, 0);
    whatDoIHave(moreMoney);

    // If the above is substitution, then so is this
    Object cat = new Cat();
    whatDoIHave(cat);
  }
  private static void whatDoIHave(Object money) {
    // Does not compile for Java, as Object does not have the method formalString(), but
    // would for non-statically-typed languages
    // However, while NewZealandMoney has a formalString() method, Cat probably does not,
    // so even if this were allowed, it would fail when "substituting" a Cat
    System.out.println("I have " + money.formalString());
  }
}
```

- Assignment is not substitution in the sense required for LSP to apply

(non) Example

```
public interface Money {
  String toString();
  String formalString();
}
public class DollarMoney {
  private int _dollars;
  private int _cents;
  public String toString() {
    return "$" + _dollars + "." + _cents;
  }
  public String formalString() {
    return _dollars + "." + _cents;
  }
}
public class NewZealandMoney extends DollarMoney implements Money {
  public NewZealandCurrency(int cents, int dollars) { ... }
  public String formalString() {
    return super.formalString() + " NZD";
  }
}
public class AussieMoney extends DollarMoney implements Money {
  public AussieCurrency(int cents, int dollars) { ... }
  public String formalString() {
    return super.formalString() + " AUD";
  }
}
```

Where substitution occurs


```
class Client {
  public static void main(String[] args) {
    NewZealandMoney nzm = new NewZealandMoney(100, 0);
    whatDoIHave(nzm);
  }
  private static void whatDoIHave(Money money) {
    // Money object expected (not DollarMoney)
    System.out.println("I have " + money.formalString());
  }
}
```

Alternative without inheritance

```
public interface Money {
  // As before
}
public class DollarMoney {
  // As before
}
public class NewZealandMoney implements Money {
  private DollarMoney _primitiveMoney; // delegate
  public NewZealandCurrency(int cents, int dollars) { ... }
  public String formalString() {
    return _primitiveMoney.formalString() + " NZD"; // "forward" or delegation
  }
  public String toString() {...}
}
public class AussieMoney implements Money {
  private DollarMoney _primitiveMoney; // delegate
  public AussieCurrency(int cents, int dollars) { ... }
  public String formalString() {
    return _primitiveMoney.formalString() + " AUD"; // "forward" or delegation
  }
  public String toString() {...}
}
```

- favor object composition over class inheritance ("Design Patterns", Gang of Four)
- If we don't expect substitution then inheritance not needed
- Can still substitute where Money expected
- (but note duplicate code)

Example continued

- when would it make sense to substitute a `AussieMoney` object for `DollarMoney` object? — and yet no obvious problem with use of `extends`
- two “uses” of inheritance
  - reuse code without copying
  - increase reusability of context by indicating type relationship

  `extends` does both
- LSP applies to type relationship

LSP and Alterability

- LSP is about designs that allow change in behaviour through substitution without requiring the code using the parent—the *context*—to change $\Rightarrow$ reducing $\mathtt{C}$.

LSP and Design Patterns

- Any design pattern that relies on inheritance, or rather polymorphism, (pretty much all of them) is based on LSP

LSP Conclusion

- Provides actionable criteria to determine whether “inheritance” has been used “properly”
- For LSP, “properly” means subtype (hence “substitution”)

Key Points

- The SOLID principles are considered important to good design by many in industry, and so should be understood if only because you are expected to
- More to come. . .